JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Class FileSystems

java.lang.Object
    java.nio.file.FileSystems

```
public final class FileSystems
extends Object
```

Factory methods for file systems. This class defines the `getDefault` method to get the
default file system and factory methods to construct other types of file systems.

The first invocation of any of the methods defined by this class causes the default `provider` to be loaded. The default provider, identified by the URI scheme "file", creates the `FileSystem` that provides access to the file systems accessible to the Java virtual machine. If the process of loading or initializing the default provider fails then an unspecified error is thrown.

The first invocation of the `installedProviders` method, by way of invoking any of the `newFileSystem` methods defined by this class, locates and loads all installed file system providers. Installed providers are loaded using the service-provider loading facility defined by the `ServiceLoader` class. Installed providers are loaded using the system class loader. If the system class loader cannot be found then the extension class loader is used; if there is no extension class loader then the bootstrap class loader is used. Providers are typically installed by placing them in a JAR file on the application class path or in the extension directory, the JAR file contains a provider-configuration file named `java.nio.file.spi.FileSystemProvider` in the resource directory `META-INF/services`, and the file lists one or more fully-qualified names of concrete subclass of `FileSystemProvider` that have a zero argument constructor. The ordering that installed providers are located is implementation specific. If a provider is instantiated and its `getScheme` returns the same URI scheme of a provider that was previously instantiated then the most recently instantiated duplicate is discarded. URI schemes are compared without regard to case. During construction

a provider may safely access files associated with the default provider but care needs to be taken to avoid circular loading of other installed providers. If circular loading of installed providers is detected then an unspecified error is thrown.

This class also defines factory methods that allow a `ClassLoader` to be specified when locating a provider. As with installed providers, the provider classes are identified by placing the provider configuration file in the resource directory `META-INF/services`.

If a thread initiates the loading of the installed file system providers and another thread invokes a method that also attempts to load the providers then the method will block until the loading completes.

Since:

`1.7`

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static **FileSystem**	**getDefault**()	Returns the default `FileSystem`.
static **FileSystem**	**getFileSystem**(**URI** uri)	Returns a reference to an existing `FileSystem`.
static **FileSystem**	**newFileSystem**(**Path** path, **ClassLoader** loader)	Constructs a new `FileSystem` to access the contents of a file as a file system.
static **FileSystem**	**newFileSystem**(**URI** uri, **Map**<**String**,?> env)	Constructs a new file system that is identified by a **URI**
static **FileSystem**	**newFileSystem**(**URI** uri, **Map**<**String**,?> env, **ClassLoader** loader)	Constructs a new file system that is identified by a **URI**

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Method Detail

getDefault

```
public static FileSystem getDefault()
```

Returns the default `FileSystem`. The default file system creates objects that provide access to the file systems accessible to the Java virtual machine. The *working directory* of the file system is the current user directory, named by the system property `user.dir`. This allows for interoperability with the `java.io.File` class.

The first invocation of any of the methods defined by this class locates the default `provider` object. Where the system property `java.nio.file.spi.DefaultFileSystemProvider` is not defined then the default provider is a system-default provider that is invoked to create the default file system.

If the system property `java.nio.file.spi.DefaultFileSystemProvider` is defined then it is taken to be a list of one or more fully-qualified names of concrete provider classes identified by the URI scheme `"file"`. Where the property is a list of more than one name then the names are separated by a comma. Each class is loaded, using the system class loader, and instantiated by invoking a one argument constructor whose formal parameter type is `FileSystemProvider`. The providers are loaded and instantiated in the order they are listed in the property. If this process fails or a provider's scheme is not equal to `"file"` then an unspecified error is thrown. URI schemes are normally compared without regard to case but for the default provider, the scheme is required to be `"file"`. The first provider class is instantiated by invoking it with a reference to the system-default provider. The second provider class is instantiated by invoking it with a reference to the first provider instance. The third provider class is instantiated by invoking it with a reference to the second instance, and so on. The last provider to be instantiated becomes the default provider; its `getFileSystem` method is invoked with the URI `"file:///"` to get a reference to the default file system.

Subsequent invocations of this method return the file system that was returned by the first invocation.

Returns:

`the default file system`

getFileSystem

```
public static FileSystem getFileSystem(URI uri)
```

Returns a reference to an existing `FileSystem`.

This method iterates over the `installed` providers to locate the provider that is identified by the URI `scheme` of the given URI. URI schemes are compared without regard to case. The exact form of the URI is highly provider dependent. If found, the provider's `getFileSystem` method is invoked to obtain a reference to the `FileSystem`.

Once a file system created by this provider is `closed` it is provider-dependent if this method returns a reference to the closed file system or throws `FileSystemNotFoundException`. If the provider allows a new file system to be created with the same URI as a file system it previously created then this method throws the exception if invoked after the file system is closed (and before a new instance is created by the `newFileSystem` method).

If a security manager is installed then a provider implementation may require to check a permission before returning a reference to an existing file system. In the case of the `default` file system, no permission check is required.

Parameters:

`uri - the URI to locate the file system`

Returns:

`the reference to the file system`

Throws:

`IllegalArgumentException - if the pre-conditions for the uri parameter are not met`

`FileSystemNotFoundException - if the file system, identified by the URI, does not exist`

`ProviderNotFoundException - if a provider supporting the URI scheme is not installed`

`SecurityException - if a security manager is installed and it denies an unspecified permission`

newFileSystem

```
public static FileSystem newFileSystem(URI uri,
                                       Map<String,?> env)
                                throws IOException
```

Constructs a new file system that is identified by a `URI`

This method iterates over the `installed` providers to locate the provider that is identified by the URI `scheme` of the given URI. URI schemes are compared without regard to case. The exact form of the URI is highly provider dependent. If found, the provider's `newFileSystem(URI,Map)` method is invoked to construct the new file system.

Once a file system is closed it is provider-dependent if the provider allows a new file system to be created with the same URI as a file system it previously created.

Usage Example: Suppose there is a provider identified by the scheme "memory" installed:

```
Map<String,String> env = new HashMap<>();
env.put("capacity", "16G");
env.put("blockSize", "4k");
FileSystem fs = FileSystems.newFileSystem(URI.create("memory:///?name=logfs"),
```

Parameters:

uri - the URI identifying the file system

env - a map of provider specific properties to configure the file system; may be empty

Returns:

a new file system

Throws:

IllegalArgumentException - if the pre-conditions for the uri parameter are not met, or the env parameter does not contain properties required by the provider, or a property value is invalid

FileSystemAlreadyExistsException - if the file system has already been created

ProviderNotFoundException - if a provider supporting the URI scheme is not installed

IOException - if an I/O error occurs creating the file system

SecurityException - if a security manager is installed and it denies an unspecified permission required by the file system provider implementation

newFileSystem

```
public static FileSystem newFileSystem(URI uri,
                                       Map<String,?> env,
                                       ClassLoader loader)
                                throws IOException
```

Constructs a new file system that is identified by a URI

This method first attempts to locate an installed provider in exactly the same manner as the newFileSystem(URI,Map) method. If none of the installed providers support the URI scheme then an attempt is made to locate the

provider using the given class loader. If a provider supporting the URI scheme is located then its `newFileSystem(URI,Map)` is invoked to construct the new file system.

Parameters:

`uri - the URI identifying the file system`

`env - a map of provider specific properties to configure the file system; may be empty`

`loader - the class loader to locate the provider or null to only attempt to locate an installed provider`

Returns:

`a new file system`

Throws:

`IllegalArgumentException - if the pre-conditions for the uri parameter are not met, or the env parameter does not contain properties required by the provider, or a property value is invalid`

`FileSystemAlreadyExistsException - if the URI scheme identifies an installed provider and the file system has already been created`

`ProviderNotFoundException - if a provider supporting the URI scheme is not found`

`ServiceConfigurationError - when an error occurs while loading a service provider`

`IOException - an I/O error occurs creating the file system`

`SecurityException - if a security manager is installed and it denies an unspecified permission required by the file system provider implementation`

newFileSystem

```
public static FileSystem newFileSystem(Path path,
                                       ClassLoader loader)
                                throws IOException
```

Constructs a new `FileSystem` to access the contents of a file as a file system.

This method makes use of specialized providers that create pseudo file systems where the contents of one or more files is treated as a file system.

This method iterates over the `installed` providers. It invokes, in turn, each provider's `newFileSystem(Path,Map)` method with an empty map. If a provider returns a file system then the iteration terminates and the file system is returned. If none of the installed providers return a `FileSystem` then an attempt is made to locate the provider using the given class loader. If

a provider returns a file system then the lookup terminates and the file system is returned.

Parameters:

`path - the path to the file`

`loader - the class loader to locate the provider or null to only attempt to locate an installed provider`

Returns:

`a new file system`

Throws:

`ProviderNotFoundException - if a provider supporting this file type cannot be located`

`ServiceConfigurationError - when an error occurs while loading a service provider`

`IOException - if an I/O error occurs`

`SecurityException - if a security manager is installed and it denies an unspecified permission`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.